

Localized Artificial Intelligence and API Windows Setup



By Rui Yang

Purpose:

The purpose of this lab is to set up a locally hosted LLM that can both be accessed through the command-line interface and through a web page. It should be able to run without access to the internet and accessible from anyone that is connected to the local network.

Background Information:

AI has become increasingly prevalent in the daily lives of everyone, and as its credibility and reliability increase, it is being incorporated in corporate settings. Artificial Intelligence utilizes different technologies such as Machine Learning and neural networks to function. Machine learning is especially important. Machine learning splits into various learning methods, and the one Artificial Intelligence utilizes is unsupervised learning, where it scans through incredibly large amounts of unnamed and unlabeled data and finds similarities, structure, and patterns from the data, effectively “learning” from the information. It then goes through a supervised fine-tuning process to improve the effectiveness and accuracy of its responses. After these steps, it is able to make predictions and analysis without human hard coding and then generate a response to user input based off the patterns that it recognizes from its training (learning).

However, as the age of AI continues to grow and as the usage of online AIs has continued to surge, it has struggled to be incorporated into other larger systems or companies like school districts because it would be leaking private data to the companies controlling/hosting the AI as whenever a user enters a prompt, that information is passed to the company hosting the AI and then processed at their data centers, which allows the AI company access to the data entered. This issue is solved with the release of open-source Large Language Models (LLMs) by the big AI companies, which allowed different companies to enjoy basic level querying and Q&A with less developed models while hosting their own LLM locally on their own networks and ensure that their own private data does not get leaked to other companies.

This imposes another issue. Large, localized AI models are very resource intensive, requiring tens if not hundreds of gigabytes of ram to function, and in order to run them effectively with enough speed for a rapid response, most of the time they are ran fully off of the VRAM inside of Graphic Processing Units (GPUs) as VRAM provides AI with multiplicatively increasing

speeds. This poses the issue of high costs of deploying a locally hosted LLM and is oftentimes not accessible to smaller scaled companies. For example, the current highest ram consumption open-source model is DeepSeek's V3 model with 671b parameters utilizing almost 2 terabytes of VRAM to function correctly with high precision. This is only realistic with large quantity NVIDIA H100 or H200 clusters (32 H100s or 16 H200s). Because of this, even if a company were to run a localized model, they would profit from having a centrally hosted LLM off of a datacenter or some GPU clusters, and then hosting the AI through a local API that allows access from the local network. This method minimizes costs while maximizing coverage, allowing the entire companies workers to have access to a reliable, localized, and private LLM that will not leak private data to other companies.

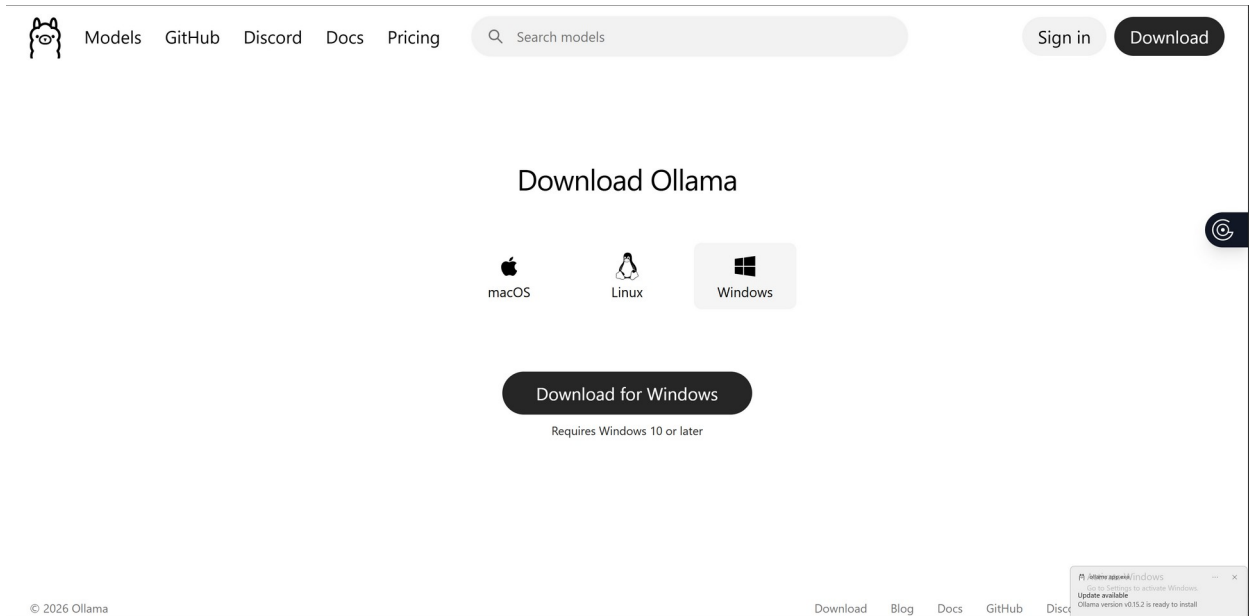
LAB SUMMARY:

In this lab, we set up and ran a small scaled, local large language model (LLM) using a combination of Docker, and Ollama. We establish a localized ai without the need to access the internet and being able to secure all information inputted in the AI. We also managed to input knowledge bases into the AI to have it respond according to trustworthy sources. In addition, we managed to utilize a pre-existing Web API from OpenWebUI to make it accessible to the entire local CTE network branch.

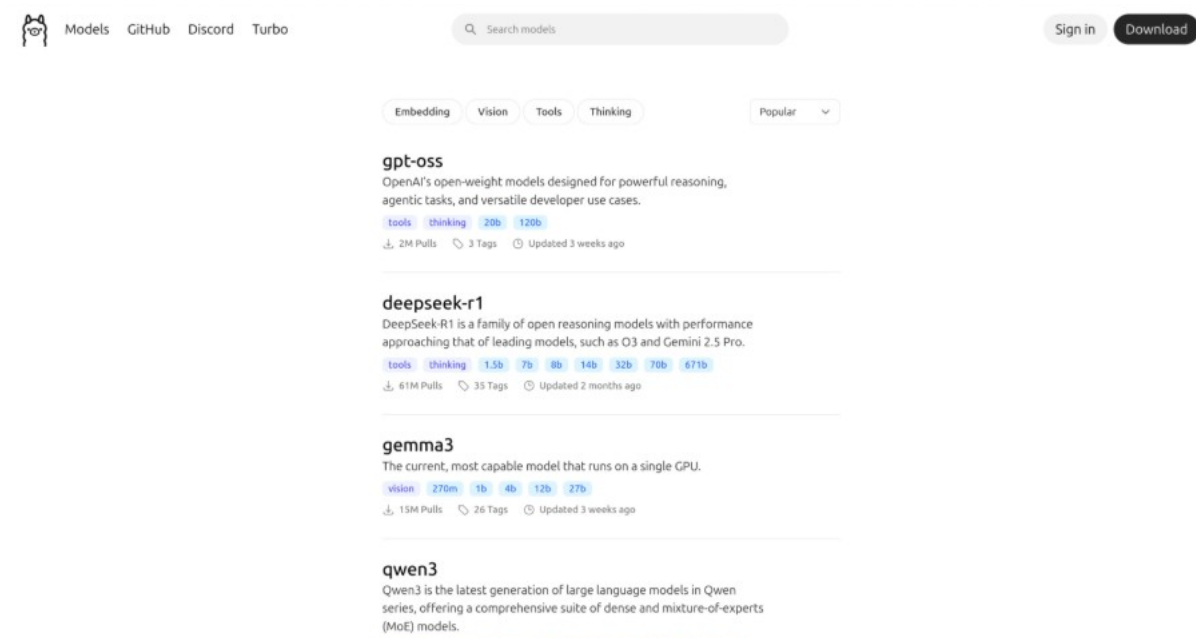
REQUIREMENTS: A DESKTOP RUNNING WINDOWS 11 AND HAVE VARIABLY (BUT ENOUGH) RAM TO RUN THE MODEL OF CHOICE. PLEASE CHECK FOR COMPATIBILITY BEFORE PROCEEDING.

Steps and Configuration:

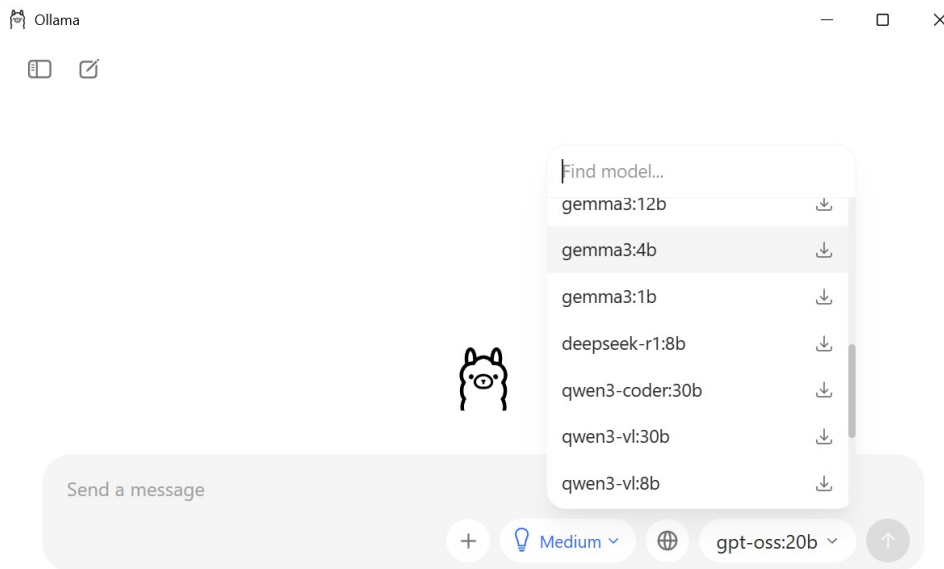
Proceed to the web browser and download ollama at [ollama download](#) with their Windows Download option:



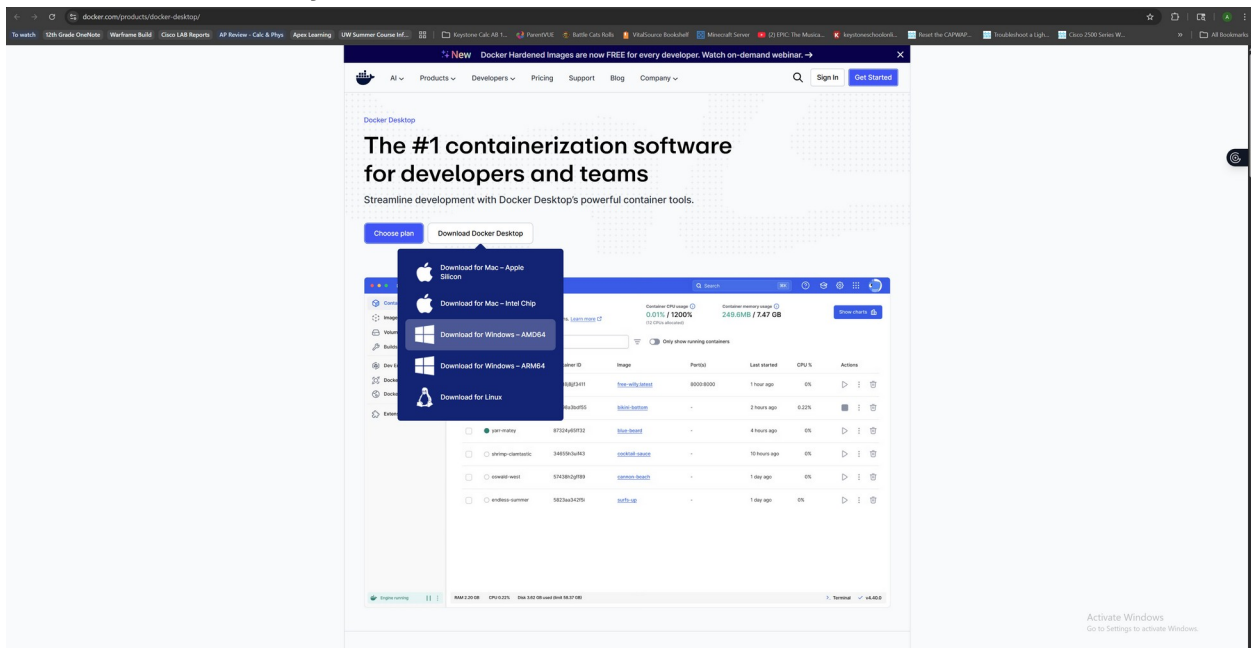
After installing ollama, look through the available ai models offered at ollama and choose one from the [search](#) feature on ollama that can be run off your v-ram.



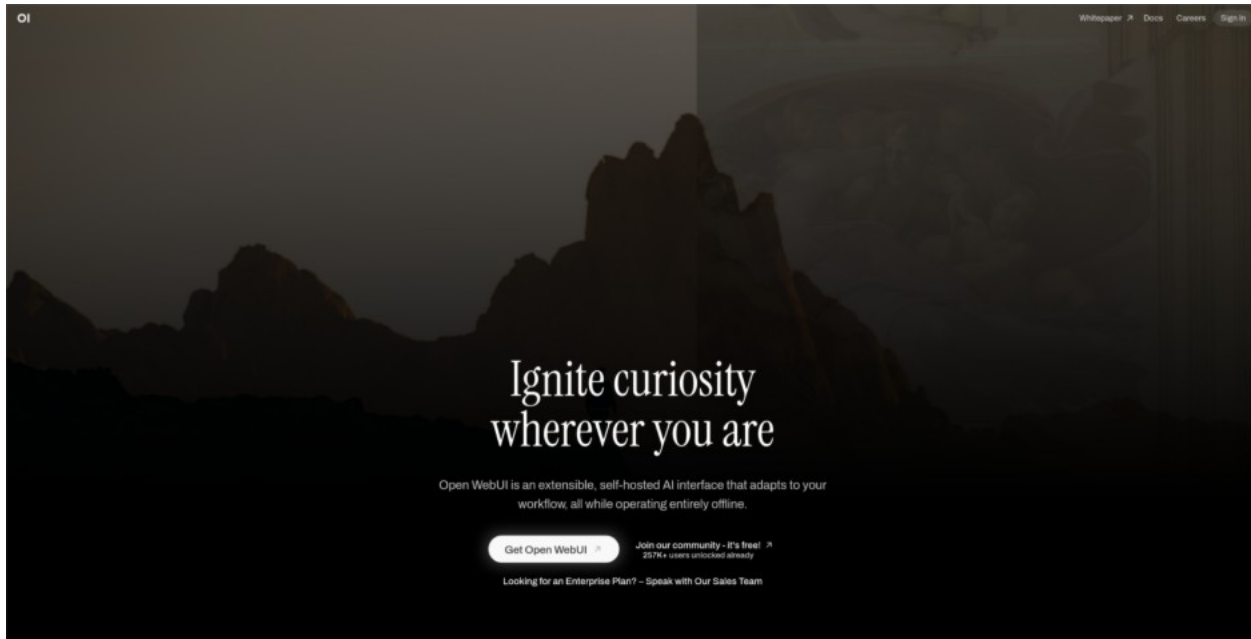
Open up ollama and download the AI model. An example is shown trying to download gemma3:4b:



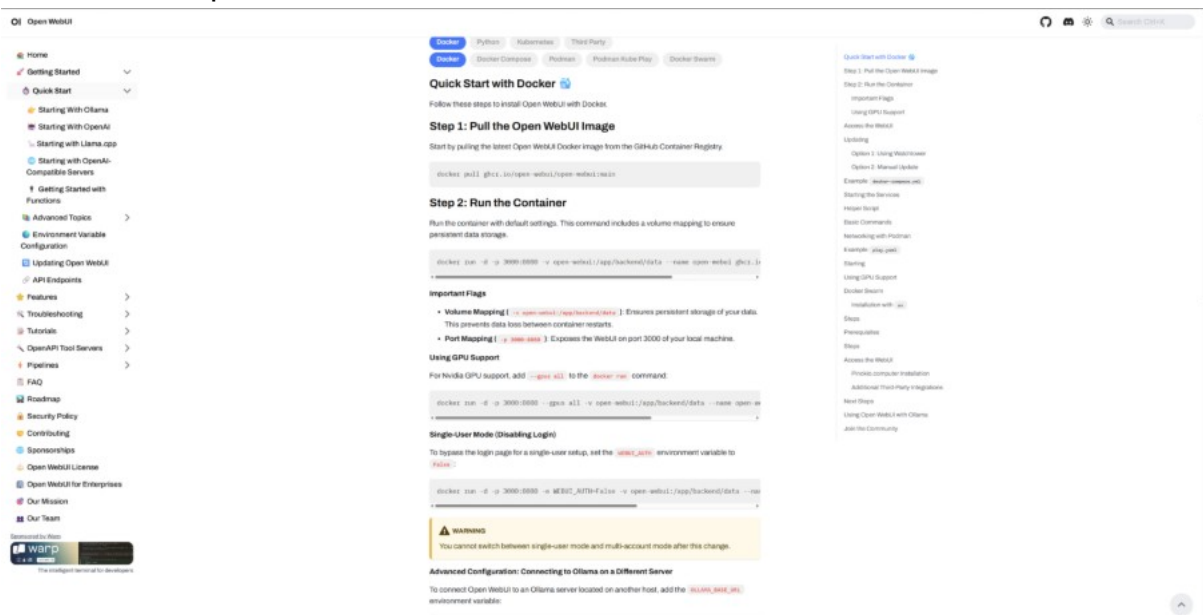
Install Docker on your machine through their website, and choose the version based off of your device



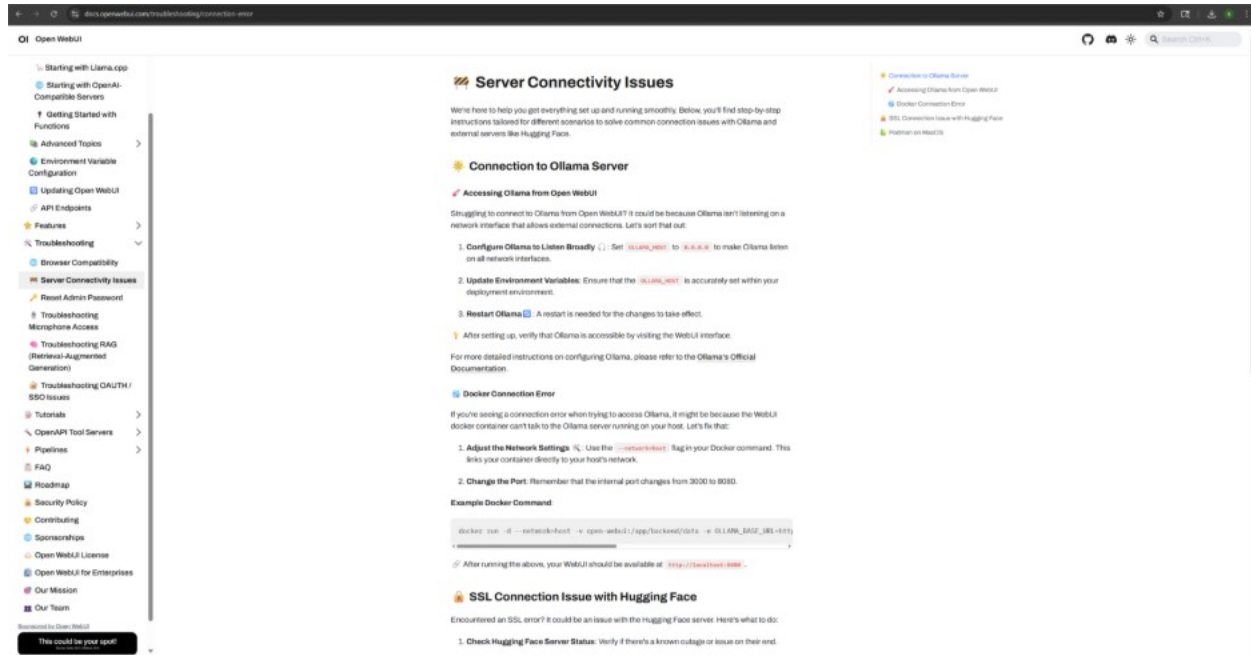
Install OpenWebUI through their official website and open the [doc](#) located on top right:



Navigate to the “Quick start” tab inside the docs and follow the instructions to download OpenWebUI



Run the container with the “-network=host” flag in order to guarantee network connectivity. You can find the command inside the troubleshooting tab, under “Server Connectivity Issues”.



Access your WebUi at “*private.i.p.address:8080*” and set up your admin account with your email and password, then navigate to the admin panel.

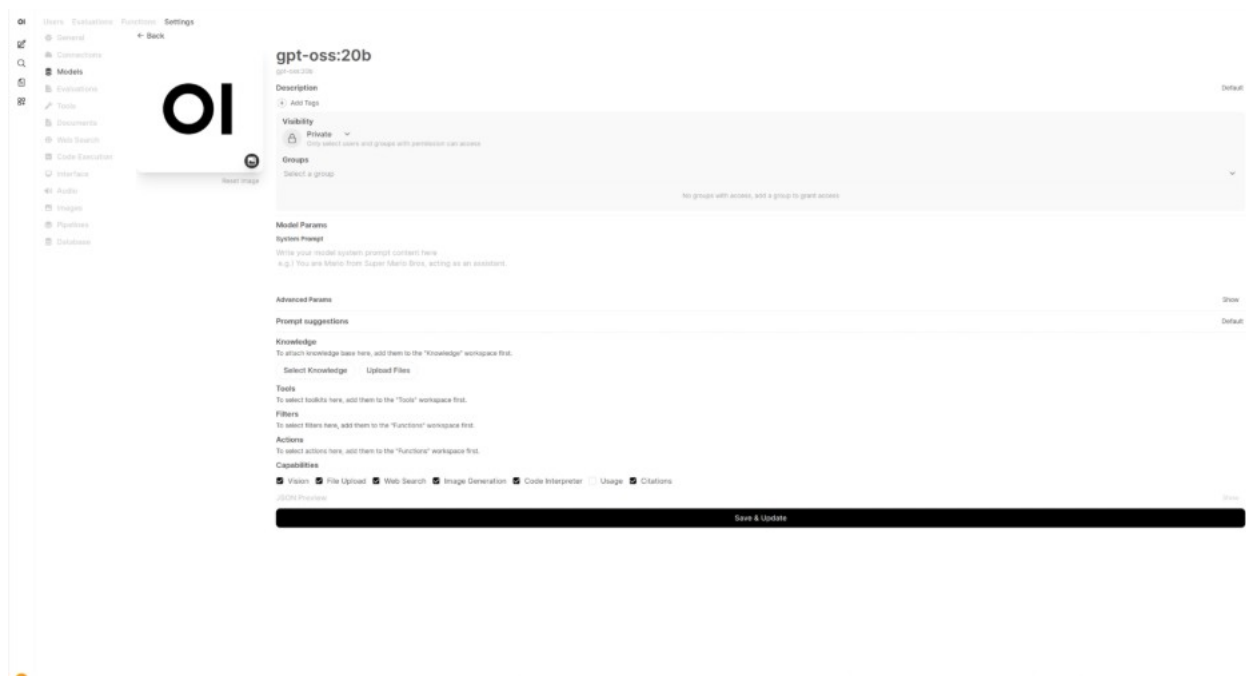


Create Users for the people using this service with the “+” on the top left

The screenshot displays a web application interface for managing users. At the top, there is a navigation bar with tabs for 'Users', 'Evolution', 'Functions', and 'Settings'. Below this, a sidebar on the left contains icons for 'Overview', 'Groups', and 'Users'. The main content area is titled 'Users 1' and features a table with the following columns: 'ROLE', 'NAME', 'EMAIL', 'LAST ACTIVE', 'CREATED AT', and 'AUTH ID'. A single user is listed with the role 'ADMIN', name 'Admin', email 'exampleemail2@gmail.com', and created at 'September 5, 2025'. A '+' icon is visible in the top right corner of the table. Below the table, a modal dialog titled 'Add User' is open, showing a form with fields for 'Role' (set to 'ADMIN'), 'Name' (set to 'User1'), and 'Email' (set to 'exampleemail2@gmail.com'). The modal also includes a 'Password' field with a strength indicator and a 'Save' button.

ROLE	NAME	EMAIL	LAST ACTIVE	CREATED AT	AUTH ID
ADMIN	Admin	exampleemail2@gmail.com	a few seconds ago	September 5, 2025	

Navigate to the “Settings” bar at the top, then choose “models”, and edit the current available models, changing the visibility to public instead of private



Everything is now complete!

Problems:

As this was a newly introduced concept, we started with a very basic knowledge of how LLMs functioned, but we needed to discover open-sourced LLMs that were accessible to us and learned how to host them locally on our PCs. There were also some issues with running docker, where it wouldn't be able to run the OpenWebUI page or the OpenWebUI , even though all the configurations were correct. There was also an underlying requirement for the OpenWebUI to be able to access ollama services and pull AIs, which was to have ollama downloaded and running on the PC at the same time.

Conclusions:

We were able to host a localized LLM that was accessible across the local network. It was able to run offline, and all the information it was using was saved locally without being sent to other companies. We also learned how to input additional information for the AI to use as context to answer some of the questions more accurately and truthfully. With a machine that has enough processing power and speed, we would be able to host a good local AI with a large training dataset and enough trustworthy context for it to use,

we can reliably ask it questions and use it to help us troubleshoot and compile data.